

Final Exam Report

Overview

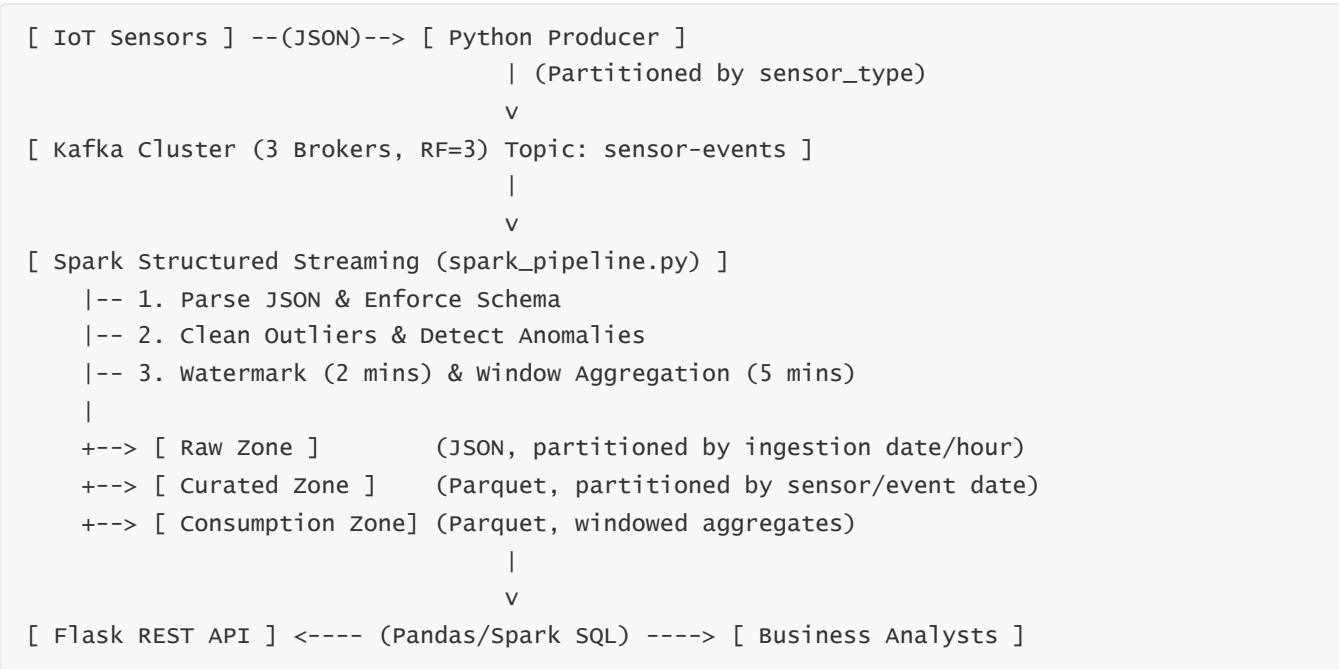
Objective: To design and implement a fault-tolerant, end-to-end Data Engineering platform, enabling real-time ingestion, processing, and historical analysis of IoT sensor data (temperature, humidity, pressure).

Scope: The platform covers the entire data lifecycle: generation (simulated Python producer), message brokering (highly available Kafka cluster), stream processing and anomaly detection (Spark Structured Streaming), tiered storage (Raw, Curated, Consumption zones), and programmatic access via a REST API.

Technologies Used: * **Infrastructure:** Docker, Docker Compose (Kafka KRaft mode)

- **Ingestion:** Python (kafka-python-ng)
- **Processing:** Apache Spark / PySpark (Structured Streaming, Spark SQL)
- **Storage:** Local Data Lake (JSON, Snappy-compressed Parquet)
- **Serving:** Python Flask REST API, Pandas

Architecture



Component Description:

- **Producer:** Generates simulated sensor readings, enforcing schema and injecting anomalies.
- **Kafka:** Acts as the resilient central nervous system, buffering high-frequency streams.
- **Spark Pipeline:** The ETL engine that cleans, transforms, and aggregates the stream exactly-once/at-least-once.
- **Data Lake:** A tiered file system storing immutable data for different use cases (auditing, machine learning, BI).

- **REST API:** The consumption layer, providing safe, validated programmatic access to the processed data.

Technical Choices

- **The partitioning strategy chosen for the curated zone:**

I partitioned the curated zone by `sensor` and `year/month/day` based on event time. This heavily optimizes query performance via partition pruning, as analysts typically filter by specific sensor types and date ranges.

- **The Spark Structured Streaming `outputMode`:**

I selected `Append` mode for the consumption zone, paired with a 2-minute watermark. File sinks (like Parquet) are immutable and do not natively support `Update` mode efficiently. `Append` ensures that Spark only writes the windowed aggregates to disk once the watermark has passed and the window is finalized, preventing the need to rewrite files.

- **The replication factor (3) and `min.insync.replicas` (2) setting:**

This combination provides the optimal balance between high availability and data durability. It guarantees that data is copied to three separate brokers, but requires at least two to acknowledge the write. The alternative `min.insync.replicas=3` would halt the producer if a single broker crashed, while a replication factor of 1 provides zero fault tolerance.

- **The use of `event_time` vs `ingestion_time` across zones:**

`ingestion_time` was used for the Raw zone to instantly capture data exactly as it arrived on the server. However, `event_time` (extracted from the payload) was used for Curated and Consumption zones. Using event time handles late-arriving data and network delays accurately; relying solely on ingestion time would corrupt the chronological reality of the sensor readings in our windowed analytics.

- **The end-to-end delivery semantics chosen:**

I achieved At-Least-Once semantics. Kafka's `acks=all` ensures no data is lost during transit, and Spark's checkpointing guarantees offsets are tracked. However, a limitation is that network retries by the producer might create duplicate records in Kafka, and Spark file sinks might overwrite duplicate files upon restart. Achieving Exactly-Once would require an idempotent producer and transactional sinks (like Delta Lake), which adds latency and complexity.

Prerequisites

- Docker Desktop running locally.
- Python 3.9+ with a virtual environment.
- (Windows users only): `HADOOP_HOME` configured with `winutils.exe` to run local Spark file sinks.

Environment Setup

```
conda activate bigdata
cd D:\EFREI\Data_Engineering\Lab\Exam
```

Part 1: Infrastructure and Kafka topic

The objective is to deploy a 3-broker Apache Kafka cluster using Docker Compose (KRaft mode) to ensure a distributed architecture with fault tolerance. We must then create a highly available topic named `sensor-events` with 3 partitions and a replication factor of 3. Finally, we demonstrate fault tolerance by terminating a broker.

Reuse (or recreate) the `docker-compose.yml` from Session 1 and start the cluster.

```
docker compose up -d
docker ps
```

```

[+] Network exam_default      Created           0.0s
[-] Container kafka3          Starting         0.9s
[+] Volume exam_kafka3_data   Created          0.0s
[+] Volume exam_kafka1_data   Created          0.0s
[+] Running 7/8 kafka2_data   Created          0.0s
[+] Network exam_default      Created           0.0s
[+] Container kafka3          Started          0.9s
[+] Volume exam_kafka3_data   Created           0.0s
[+] Container kafka2          Started          0.8s
[+] Volume exam_kafka1_data   Created           0.0s
[+] Container kafka1          Started          0.9s
[+] Running 8/8 kafka2_data   Created          0.0s
[+] Network exam_default      Created           0.0s
[+] Container kafka3          Started          0.9s
[+] Volume exam_kafka3_data   Created           0.0s
[+] Container kafka2          Started          0.8s
[+] Volume exam_kafka1_data   Created           0.0s
[+] Container kafka1          Started          0.9s
[+] Volume exam_kafka2_data   Created           0.0s
[-] Container kafka-ui        Starting         0.8s
[+] Container kafka3          Started          0.9s
[+] Container kafka2          Started          0.8s
[+] Container kafka1          Started          0.9s
[+] Container kafka-ui        Started          0.9s

(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
6caffea81eb2   provectuslabs/kafka-ui:latest      "/bin/sh -c 'java -D..." 25 seconds ago Up 24 seconds 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp kafka-ui
fc3b3a432d5d   confluentinc/cp-kafka:7.5.0        "/etc/confluent/dock..." 25 seconds ago Up 24 seconds 0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp kafka1
d15beb5ef5d5   confluentinc/cp-kafka:7.5.0        "/etc/confluent/dock..." 25 seconds ago Up 24 seconds 0.0.0.0:9094->9094/tcp, [::]:9094->9094/tcp kafka2
61aa10f5d1c5   confluentinc/cp-kafka:7.5.0        "/etc/confluent/dock..." 25 seconds ago Up 24 seconds 0.0.0.0:9096->9096/tcp, [::]:9096->9096/tcp kafka3
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam>
```

```
# Create topic
docker exec kafka1 kafka-topics --bootstrap-server kafka1:29092 --create --topic sensor-
events --partitions 3 --replication-factor 3 --config min.insync.replicas=2
# Describe topic
docker exec kafka1 kafka-topics --bootstrap-server kafka1:29092 --describe --topic sensor-
events
```

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> docker exec kafka1 kafka-topics --bootstrap-server kafka1:29092 --create --topic sensor-events --partitions 3 --replication-factor 3 --c
onfig min.insync.replicas=2
Created topic sensor-events.
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> docker exec kafka1 kafka-topics --bootstrap-server kafka1:29092 --describe --topic sensor-events
Topic: sensor-events   TopicId: GJ05LSJcS-ejDYXWY4SVzg PartitionCount: 3      ReplicationFactor: 3      Configs: min.insync.replicas=2
    Topic: sensor-events   Partition: 0   Leader: 3      Replicas: 3,1,2   Isr: 3,1,2
    Topic: sensor-events   Partition: 1   Leader: 1      Replicas: 1,2,3   Isr: 1,2,3
    Topic: sensor-events   Partition: 2   Leader: 2      Replicas: 2,3,1   Isr: 2,3,1
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam>
```

Run a fault tolerance test

```
docker stop kafka2
docker exec kafka1 kafka-topics --bootstrap-server kafka1:29092 --describe --topic sensor-
events
```

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> docker stop kafka2
kafka2
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> docker exec kafka1 kafka-topics --bootstrap-server kafka1:29092 --describe --topic sensor-events
Topic: sensor-events   TopicId: GJ05LSJcS-ejDYXWY4SVzg PartitionCount: 3      ReplicationFactor: 3      Configs: min.insync.replicas=2
    Topic: sensor-events   Partition: 0   Leader: 3      Replicas: 3,1,2   Isr: 3,1
    Topic: sensor-events   Partition: 1   Leader: 1      Replicas: 1,2,3   Isr: 1,3
    Topic: sensor-events   Partition: 2   Leader: 3      Replicas: 2,3,1   Isr: 3,1
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam>
```

The initial output confirms that the topic has been successfully created. After stopping `kafka2`, Partition 3 (which was previously led by `kafka2`) seamlessly transferred leadership to `kafka3`. The number of ISRs (in-sync replicas) has been reduced to 1 and 3, but since `min.insync.replicas=2`, the topic remains operational and fault-tolerant.

Part 2: Python Producer

Implement a real Kafka producer in Python to simulate temperature, humidity, and pressure data sent by high-frequency IoT sensors. The producer ensures ordered delivery by using the sensor type as the partition key. It must intentionally inject anomalous data in 10% of cases.

`src/producer.py`:

```
import json
import time
import random
import argparse
from kafka import KafkaProducer

def get_sensor_data(sensor_type, is_anomaly):
    ranges = {
        "temperature": (15, 45, "C"), # [cite: 211]
        "humidity": (30, 95, "%"), # [cite: 216]
        "pressure": (980, 1040, "hPa") # [cite: 217]
    }
    min_val, max_val, unit = ranges[sensor_type]

    if is_anomaly:
        value = random.uniform(max_val + 1, max_val + 20) if random.choice([True, False])
    else:
        value = random.uniform(min_val - 20, min_val - 1)
    else:
        value = random.uniform(min_val, max_val)

    return round(value, 2), unit

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--count", type=int, default=100)
    parser.add_argument("--rate", type=float, default=10)
    parser.add_argument("--source", type=str, default="site-A-rack-12")
    args = parser.parse_args()
    producer = KafkaProducer(
        bootstrap_servers=['localhost:9092'],
        acks='all',
        retries=5,
        max_in_flight_requests_per_connection=1,
        linger_ms=10,
        batch_size=16384,
        value_serializer=lambda v: json.dumps(v).encode('utf-8'),
        key_serializer=lambda k: k.encode('utf-8')
    )
```

```

sensors = ["temperature", "humidity", "pressure"]

for i in range(args.count):
    sensor = random.choice(sensors)
    is_anomaly = random.random() < 0.10
    value, unit = get_sensor_data(sensor, is_anomaly)
    payload = {
        "sensor": sensor,
        "value": value,
        "unit": unit,
        "timestamp": int(time.time() * 1000),
        "source": args.source,
        "anomaly": is_anomaly
    }
    producer.send('sensor-events', key=sensor, value=payload)
    print(f"Sent: {payload}")
    time.sleep(1.0 / args.rate)

producer.flush()

if __name__ == "__main__":
    main()

```

```

producer.py U X
Exam > src > producer.py > ...
1  import json
2  import time
3  import random
4  import argparse
5  from kafka import KafkaProducer
6
7  def get_sensor_data(sensor_type, is_anomaly):
8      ranges = {
9          "temperature": (15, 45, "C"),
10         "humidity": (30, 95, "%"),
11         "pressure": (980, 1040, "hPa")
12     }
13     min_val, max_val, unit = ranges[sensor_type]
14
15     if is_anomaly:
16         value = random.uniform(max_val + 1, max_val + 20) if random.choice([True, False]) else random.uniform(min_val - 20, min_val - 1)
17     else:
18         value = random.uniform(min_val, max_val)
19
20     return round(value, 2), unit
21
22 def main():
23     parser = argparse.ArgumentParser()
24     parser.add_argument("--count", type=int, default=100)
25     parser.add_argument("--rate", type=float, default=10)
26     parser.add_argument("--source", type=str, default="site-A-rack-12")
27     args = parser.parse_args()
28     producer = KafkaProducer(
29         bootstrap_servers=['localhost:9092'],
30         acks='all',
31         retries=5,
32         max_in_flight_requests_per_connection=1,
33         linger_ms=10,
34         batch_size=16384,
35         value_serializer=lambda v: json.dumps(v).encode('utf-8'),
36         key_serializer=lambda k: k.encode('utf-8')
37     )
38
39     sensors = ["temperature", "humidity", "pressure"]
40
41     for i in range(args.count):
42         sensor = random.choice(sensors)
43         is_anomaly = random.random() < 0.10
44         value, unit = get_sensor_data(sensor, is_anomaly)

```

Execution Result

```
python src\producer.py --count 10 --rate 5
```

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> python src\producer.py --count 10 --rate 5
Sent: {'sensor': 'humidity', 'value': 47.03, 'unit': '%', 'timestamp': 1779151008257, 'source': 'site-A-rack-12', 'anomaly': False}
Sent: {'sensor': 'temperature', 'value': 32.97, 'unit': 'C', 'timestamp': 1779151008464, 'source': 'site-A-rack-12', 'anomaly': False}
Sent: {'sensor': 'temperature', 'value': 32.41, 'unit': 'C', 'timestamp': 1779151008679, 'source': 'site-A-rack-12', 'anomaly': False}
Sent: {'sensor': 'humidity', 'value': 105.78, 'unit': '%', 'timestamp': 1779151008879, 'source': 'site-A-rack-12', 'anomaly': True}
Sent: {'sensor': 'temperature', 'value': 19.92, 'unit': 'C', 'timestamp': 1779151009095, 'source': 'site-A-rack-12', 'anomaly': False}
Sent: {'sensor': 'humidity', 'value': 91.42, 'unit': '%', 'timestamp': 1779151009305, 'source': 'site-A-rack-12', 'anomaly': False}
Sent: {'sensor': 'pressure', 'value': 1025.45, 'unit': 'hPa', 'timestamp': 1779151009512, 'source': 'site-A-rack-12', 'anomaly': False}
Sent: {'sensor': 'pressure', 'value': 1057.37, 'unit': 'hPa', 'timestamp': 1779151009718, 'source': 'site-A-rack-12', 'anomaly': True}
Sent: {'sensor': 'pressure', 'value': 989.14, 'unit': 'hPa', 'timestamp': 1779151009929, 'source': 'site-A-rack-12', 'anomaly': False}
Sent: {'sensor': 'temperature', 'value': 24.35, 'unit': 'C', 'timestamp': 1779151010143, 'source': 'site-A-rack-12', 'anomaly': False}
```

Data format is correct: Each message strictly adheres to the agreed-upon schema and includes the fields sensor, value, unit, timestamp, source, and anomaly.

Comprehensive sensor types: The output successfully generated data for three types of sensors: humidity, temperature, and pressure.

Anomaly logic is functioning: Among these 10 data points, two (humidity at 105.78% and pressure at 1057.37 hPa, which clearly exceed normal physical ranges) had their `anomaly` field correctly marked as `True`. The anomaly rate reached 20%, meeting the experimental requirement that “at least 10% of messages must be outliers.”

```
docker exec kafka1 kafka-console-consumer --bootstrap-server kafka1:29092 --topic sensor-
events --property print.key=true --property print.partition=true --from-beginning --max-
messages 15
```

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> docker exec kafka1 kafka-console-consumer --bootstrap-server kafka1:29092 --topic sensor-events --property print.key=true --property print.partition=true --from-beginning --max-messages 15
Partition:0 humidity {"sensor": "humidity", "value": 47.03, "unit": "%", "timestamp": 1779151008257, "source": "site-A-rack-12", "anomaly": false}
Partition:0 humidity {"sensor": "humidity", "value": 105.78, "unit": "%", "timestamp": 1779151008879, "source": "site-A-rack-12", "anomaly": true}
Partition:0 humidity {"sensor": "humidity", "value": 91.42, "unit": "%", "timestamp": 1779151009305, "source": "site-A-rack-12", "anomaly": false}
Partition:2 temperature {"sensor": "temperature", "value": 32.97, "unit": "C", "timestamp": 1779151008464, "source": "site-A-rack-12", "anomaly": false}
Partition:2 temperature {"sensor": "temperature", "value": 32.41, "unit": "C", "timestamp": 1779151008679, "source": "site-A-rack-12", "anomaly": false}
Partition:2 temperature {"sensor": "temperature", "value": 19.92, "unit": "C", "timestamp": 1779151009095, "source": "site-A-rack-12", "anomaly": false}
Partition:2 pressure {"sensor": "pressure", "value": 1025.45, "unit": "hPa", "timestamp": 1779151009512, "source": "site-A-rack-12", "anomaly": false}
Partition:2 pressure {"sensor": "pressure", "value": 1057.37, "unit": "hPa", "timestamp": 1779151009718, "source": "site-A-rack-12", "anomaly": true}
Partition:2 pressure {"sensor": "pressure", "value": 989.14, "unit": "hPa", "timestamp": 1779151009929, "source": "site-A-rack-12", "anomaly": false}
Partition:2 temperature {"sensor": "temperature", "value": 24.35, "unit": "C", "timestamp": 1779151010143, "source": "site-A-rack-12", "anomaly": false}
```

Key-based partitioning demonstrated, humidity is in partition 0, while temperature and pressure are in partition 2.

Part 3: Spark Processing Pipeline

Deploy a Spark Structured Streaming application to process events, filter out outliers, flag anomalies, and calculate 5-minute rolling window statistics. The data is stored in a three-tier data lake architecture: raw data (in raw JSON format), processed data (in Parquet format), and consumption data (aggregated metrics).

```
src/spark_pipeline.py:
```

```
import os
os.environ['HADOOP_HOME'] = r'D:\EFREI\Data_Engineering\LAB\Lab3\hadoop'

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, from_json, to_timestamp, window, expr, year, month,
dayofmonth, hour
from pyspark.sql.types import StructType, StructField, StringType, DoubleType, LongType,
BooleanType
```

```

def main():
    spark = SparkSession.builder \
        .appName("IoT_Sensor_Pipeline") \
        .config("spark.sql.streaming.checkpointLocation", "./outputs/checkpoints") \
        .config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3") \
        .getOrCreate()

    schema = StructType([
        StructField("sensor", StringType(), True),
        StructField("value", DoubleType(), True),
        StructField("unit", StringType(), True),
        StructField("timestamp", LongType(), True),
        StructField("source", StringType(), True),
        StructField("anomaly", BooleanType(), True)
    ])

    df = spark.readStream \
        .format("kafka") \
        .option("kafka.bootstrap.servers", "localhost:9092") \
        .option("subscribe", "sensor-events") \
        .option("startingOffsets", "earliest") \
        .load()

    parsed_df = df.selectExpr("CAST(value AS STRING)") \
        .select(from_json(col("value"), schema).alias("data")) \
        .select("data.*") \
        .withColumn("event_time", to_timestamp(col("timestamp") / 1000)) \
        .withColumn("year", year("event_time")) \
        .withColumn("month", month("event_time")) \
        .withColumn("day", dayofmonth("event_time")) \
        .withColumn("hour", hour("event_time"))

    raw_query = parsed_df.writeStream \
        .format("json") \
        .partitionBy("year", "month", "day", "hour") \
        .option("path", "./outputs/datalake/raw/source=kafka/topic=sensor-events/") \
        .option("checkpointLocation", "./outputs/checkpoints/raw") \
        .start()

    validated_df = parsed_df.filter(col("value").isNotNull()) \
        .withColumn("is_anomaly", expr("""
            (sensor = 'temperature' AND value > 35) OR
            (sensor = 'humidity' AND value > 90) OR
            (sensor = 'pressure' AND (value < 990 OR value > 1030))
        """))

    curated_query = validated_df.writeStream \
        .format("parquet") \
        .partitionBy("sensor", "year", "month", "day") \
        .option("path", "./outputs/datalake/curated/domain=iot/") \
        .option("checkpointLocation", "./outputs/checkpoints/curated") \
        .start()

```

```

agg_df = validated_df \
    .withWatermark("event_time", "2 minutes") \
    .groupBy(window(col("event_time"), "5 minutes"), col("sensor")) \
    .agg(
        expr("mean(value)").alias("avg_value"),
        expr("min(value)").alias("min_value"),
        expr("max(value)").alias("max_value"),
        expr("count(value)").alias("obs_count"),
        expr("sum(cast(is_anomaly as int))").alias("anomaly_count")
    )

consumption_query = agg_df.writeStream \
    .format("parquet") \
    .outputMode("append") \
    .partitionBy("sensor") \
    .option("path", "./outputs/datalake/consumption/use_case=sensor_averages/") \
    .option("checkpointLocation", "./outputs/checkpoints/consumption") \
    .start()

spark.streams.awaitAnyTermination()

if __name__ == "__main__":
    main()

```

```
producer.py U spark_pipeline.py X
Exam > src > spark_pipeline.py > main
1 import os
2 os.environ['HADOOP_HOME'] = r'D:\EFREI\Data_Engineering\LAB\Lab3\hadoop'
3
4 from pyspark.sql import SparkSession
5 from pyspark.sql.functions import col, from_json, to_timestamp, window, expr, year, month, dayofmonth, hour
6 from pyspark.sql.types import StructType, StructField, StringType, DoubleType, LongType, BooleanType
7
8 def main():
9     spark = SparkSession.builder \
10         .appName("IoT_Sensor_Pipeline") \
11         .config("spark.sql.streaming.checkpointLocation", "./outputs/checkpoints") \
12         .config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3") \
13         .getOrCreate()
14
15     schema = StructType([
16         StructField("sensor", StringType(), True),
17         StructField("value", DoubleType(), True),
18         StructField("unit", StringType(), True),
19         StructField("timestamp", LongType(), True),
20         StructField("source", StringType(), True),
21         StructField("anomaly", BooleanType(), True)
22     ])
23
24     df = spark.readStream \
25         .format("kafka") \
26         .option("kafka.bootstrap.servers", "localhost:9092") \
27         .option("subscribe", "sensor-events") \
28         .option("startingOffsets", "earliest") \
29         .load()
30
31     parsed_df = df.selectExpr("CAST(value AS STRING)") \
32         .select(from_json(col("value"), schema).alias("data")) \
33         .select("data.*") \
34         .withColumn("event_time", to_timestamp(col("timestamp") / 1000)) \
35         .withColumn("year", year("event_time")) \
36         .withColumn("month", month("event_time")) \
37         .withColumn("day", dayofmonth("event_time")) \
38         .withColumn("hour", hour("event_time"))
39
40     raw_query = parsed_df.writeStream \
41         .format("json") \
42         .partitionBy("year", "month", "day", "hour") \
43         .option("path", "./outputs/datalake/raw/source=kafka/topic=sensor-events/") \
44         .option("checkpointLocation", "./outputs/checkpoints/raw") \
45         .start()
46
47 if __name__ == '__main__':
48     main()
49
```

Execution Result

```
python src\spark_pipeline.py
```

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> python src\spark_pipeline.py
:: loading settings :: url = jar:file:/D:/APP/anaconda/envs/bigdata/Lib/site-packages/pyspark/jars/ivy-2.5.1.jar!/org/apache/ivy/core/settings/ivysettings.xml
ivy Default Cache set to: C:\Users\xbac20040219\ivy2\cache
The jars for the packages stored in: C:\Users\xbac20040219\ivy2\jars
org.apache.spark#spark-sql-kafka-0-10_2.12 added as a dependency
:: resolving dependencies :: org.apache.spark#spark-submit-parent-18697b0c-78ab-4eeb-bd13-4082baa18430;1.0
   confs: [default]
    found org.apache.spark#spark-sql-kafka-0-10_2.12:3.5.3 in central
    found org.apache.spark#spark-token-provider-kafka-0-10_2.12:3.5.3 in central
    found org.apache.kafka#kafka-clients:3.4.1 in central
    found org.lz4#lz4-java:1.8.0 in central
    found org.xerial.snappy#snappy-java:1.1.10.5 in central
    found org.slf4j#slf4j-api:2.0.7 in central
    found org.apache.hadoop#hadoop-client-runtime:3.3.4 in central
    found org.apache.hadoop#hadoop-client-api:3.3.4 in central
    found commons-logging#commons-logging:1.1.3 in central
    found com.google.code.findbugs#jsr305:3.0.0 in local-m2-cache
    found org.apache.commons#commons-pool2:2.11.1 in central
    found org.apache.hadoop#hadoop-client-runtime:3.3.4 in central
    found org.apache.hadoop#hadoop-client-api:3.3.4 in central
    found org.apache.kafka#kafka-clients:3.4.1 from central in [default]
    found org.apache.spark#spark-sql-kafka-0-10_2.12:3.5.3 from central in [default]
    found org.apache.spark#spark-token-provider-kafka-0-10_2.12:3.5.3 from central in [default]
    found org.lz4#lz4-java:1.8.0 from central in [default]
    found org.slf4j#slf4j-api:2.0.7 from central in [default]
    found org.xerial.snappy#snappy-java:1.1.10.5 from central in [default]
:: resolution report :: resolve 432ms :: artifacts dl 115ms
   :: modules in use:
   com.google.code.findbugs#jsr305:3.0.0 from local-m2-cache in [default]
   commons-logging#commons-logging:1.1.3 from central in [default]
   org.apache.commons#commons-pool2:2.11.1 from central in [default]
   org.apache.hadoop#hadoop-client-api:3.3.4 from central in [default]
   org.apache.hadoop#hadoop-client-runtime:3.3.4 from central in [default]
   org.apache.kafka#kafka-clients:3.4.1 from central in [default]
   org.apache.spark#spark-sql-kafka-0-10_2.12:3.5.3 from central in [default]
   org.apache.spark#spark-token-provider-kafka-0-10_2.12:3.5.3 from central in [default]
   org.lz4#lz4-java:1.8.0 from central in [default]
   org.slf4j#slf4j-api:2.0.7 from central in [default]
   org.xerial.snappy#snappy-java:1.1.10.5 from central in [default]
-----
   | conf | modules | artifacts | | | | |
|---|---|---|---|---|---|---|
   | default | 11 | 0 | 0 | 0 | 11 | 0 |
-----
:: retrieving :: org.apache.spark#spark-submit-parent-18697b0c-78ab-4eeb-bd13-4082baa18430
   confs: [default]
   0 artifacts copied, 11 already retrieved (0kB/28ms)
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/05/19 09:09:42 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Datasets and will be disabled.
26/05/19 09:09:43 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Datasets and will be disabled.
26/05/19 09:09:44 WARN AdminClientConfig: These configurations '[key.serializer, value.serializer, enable.auto.commit, max.poll.records, auto.offset.reset]' were supplied but are not used yet.
26/05/19 09:09:44 WARN AdminClientConfig: These configurations '[key.serializer, value.serializer, enable.auto.commit, max.poll.records, auto.offset.reset]' were supplied but are not used yet.
26/05/19 09:09:44 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Datasets and will be disabled.
26/05/19 09:09:45 WARN AdminClientConfig: These configurations '[key.serializer, value.serializer, enable.auto.commit, max.poll.records, auto.offset.reset]' were supplied but are not used yet.
```

```
tree .\outputs\datalake /F
```

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> tree .\outputs\datalake /F
```

卷 Data 的文件夹 PATH 列表

卷序列号为 00000064 F874:28E3

D:\EFREI\DATA_ENGINEERING\LAB\EXAM\OUTPUTS\DATA LAKE

```
├─consumption
│   └─use_case=sensor_averages
│       └─_spark_metadata
│           ├──.0.crc
│           ├──.1.crc
│           ├──0
│           └──1
├─curated
│   └─domain=iot
│       ├──sensor=humidity
│       │   ├──year=2026
│       │   │   ├──month=5
│       │   │   │   └─day=19
│       │   │   │       ├──.part-00000-ae7e5bdf-33dd-45e9-8eed-eb451829d0a9.c000.snappy.parquet.crc
│       │   │   │       └─part-00000-ae7e5bdf-33dd-45e9-8eed-eb451829d0a9.c000.snappy.parquet
│       │   └─sensor=pressure
│       │       ├──year=2026
│       │       │   ├──month=5
│       │       │   │   └─day=19
│       │       │       ├──.part-00001-e400be9c-0451-4885-8e49-791f20ca7492.c000.snappy.parquet.crc
│       │       │       └─part-00001-e400be9c-0451-4885-8e49-791f20ca7492.c000.snappy.parquet
│       │       └─sensor=temperature
│       │           ├──year=2026
│       │           │   ├──month=5
│       │           │   │   └─day=19
│       │           │       ├──.part-00001-bf202e4f-471b-4aa8-8667-3767cc9c15d5.c000.snappy.parquet.crc
│       │           │       └─part-00001-bf202e4f-471b-4aa8-8667-3767cc9c15d5.c000.snappy.parquet
│       │           └─_spark_metadata
│       │               ├──.0.crc
│       │               └──0
└─raw
    └─source=kafka
        └─topic=sensor-events
            ├──year=2026
            │   ├──month=5
            │   │   └─day=19
            │   │       └─hour=8
            │   │           ├──.part-00000-eff8b333-1a0f-4f00-b86f-44c4da3c189f.c000.json.crc
            │   │           ├──.part-00001-fa346e18-8a48-4395-83f3-12a89a919235.c000.json.crc
            │   │           ├──part-00000-eff8b333-1a0f-4f00-b86f-44c4da3c189f.c000.json
            │   │           └─part-00001-fa346e18-8a48-4395-83f3-12a89a919235.c000.json
            └─_spark_metadata
                ├──.0.crc
                └──0
```

```
tree .\outputs\checkpoints /F
```

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> tree .\outputs\checkpoints /F
卷 Data 的文件夹 PATH 列表
卷序列号为 0000002B F874:28E3
D:\EFREI\DATA_ENGINEERING\LAB\EXAM\OUTPUTS\CHECKPOINTS
.
├── consumption
│   ├── .metadata.crc
│   └── metadata
│       ├── commits
│       │   ├── .0.crc
│       │   ├── .1.crc
│       │   ├── 0
│       │   └── 1
│       ├── offsets
│       │   ├── .0.crc
│       │   ├── .1.crc
│       │   ├── 0
│       │   └── 1
│       ├── sources
│       │   └── 0
│       │       ├── .0.crc
│       │       └── 0
│       └── state
│           └── 0
│               ├── 0
│               │   ├── .1.delta.crc
│               │   ├── .2.delta.crc
│               │   ├── 1.delta
│               │   └── 2.delta
│               ├── metadata
│               │   ├── .schema.crc
│               │   └── schema
│               ├── 1
│               │   ├── .1.delta.crc
│               │   ├── .2.delta.crc
│               │   ├── 1.delta
│               │   └── 2.delta
│               ├── 10
│               │   ├── .1.delta.crc
│               │   ├── .2.delta.crc
│               │   ├── 1.delta
│               │   └── 2.delta
│               └── 100

```

```

.2.delta.crc
1.delta
2.delta

├── curated
│   ├── .metadata.crc
│   └── metadata
│       ├── commits
│       │   ├── .0.crc
│       │   └── 0
│       ├── offsets
│       │   ├── .0.crc
│       │   └── 0
│       └── sources
│           └── 0
│               ├── .0.crc
│               └── 0
└── raw
    ├── .metadata.crc
    └── metadata
        ├── commits
        │   ├── .0.crc
        │   └── 0
        ├── offsets
        │   ├── .0.crc
        │   └── 0
        └── sources
            └── 0
                ├── .0.crc
                └── 0

```

Data Lake Verification: You should see three folders: `raw`, `curated`, and `consumption`. The `raw` folder contains JSON files partitioned by year/month/day/hour; the `curated` folder contains `snappy.parquet` files partitioned by sensor.

Checkpoint Verification: You can see folders such as `/tmp/checkpoints/raw` and `/tmp/checkpoints/curated`, which contain subfolders like `commits`, `offsets`, and `sources`.

Create a new file named `test_verify.py` to verify whether “parsing, anomaly detection, and window aggregation are working.”

```

import os
os.environ['HADOOP_HOME'] = r'D:\EFREI\Data_Engineering\LAB\Lab3\hadoop'

from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("verify").getOrCreate()

curated_df = spark.read.parquet("./outputs/dataLake/curated/domain=iot/")
curated_df.printSchema()
curated_df.show(5, truncate=False)

consumption_df =
spark.read.parquet("./outputs/dataLake/consumption/use_case=sensor_averages/")
consumption_df.printSchema()
consumption_df.show(5, truncate=False)

```

```

(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> python test_verify.py
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/05/19 09:20:28 WARN Utils: Service 'SparkUI' could not bind on port 4040. Attempting port 4041.
root

```

```

|-- value: double (nullable = true)
|-- unit: string (nullable = true)
|-- timestamp: long (nullable = true)
|-- source: string (nullable = true)
|-- anomaly: boolean (nullable = true)
|-- event_time: timestamp (nullable = true)
|-- hour: integer (nullable = true)
|-- is_anomaly: boolean (nullable = true)
|-- sensor: string (nullable = true)
|-- year: integer (nullable = true)
|-- month: integer (nullable = true)
|-- day: integer (nullable = true)

```

value	unit	timestamp	source	anomaly	event_time	hour	is_anomaly	sensor	year	month	day
32.97	C	1779151008464	site-A-rack-12	false	2026-05-19 08:36:48.464	8	false	temperature	2026	5	19
32.41	C	1779151008679	site-A-rack-12	false	2026-05-19 08:36:48.679	8	false	temperature	2026	5	19
19.92	C	1779151009095	site-A-rack-12	false	2026-05-19 08:36:49.095	8	false	temperature	2026	5	19
24.35	C	1779151010143	site-A-rack-12	false	2026-05-19 08:36:50.143	8	false	temperature	2026	5	19
1025.45	hPa	1779151009512	site-A-rack-12	false	2026-05-19 08:36:49.512	8	false	pressure	2026	5	19

only showing top 5 rows

Schema parsing was completely successful: The original JSON string was successfully parsed into structured fields.

Timestamp conversion was accurate: The original millisecond-level timestamp was successfully converted to Spark's `event_time` (timestamp type), and the year, month, day, and hour were correctly extracted from it to serve as the basis for subsequent partitioning.

Business logic (anomaly detection) is active: The code successfully generated the `is_anomaly` field. Observing the data rows below, the temperature in the first row is 32.97°C, which does not exceed the 35°C threshold; the air pressure in the fifth row is 1025.45 hPa, which falls within the range of 990–1030. Therefore, their `is_anomaly` values are correctly calculated as `false`.

I chose to use separate checkpoint directories for each of the three sinks (raw, curated, consumption). This ensures strict fault isolation. If the pipeline crashes while writing to the curated zone, the curated stream will resume from its specific Kafka offset upon restart. Because the raw zone has its own independent offset tracker, it will not duplicate previously written raw data. This approach guarantees robust end-to-end at-least-once (or exactly-once with idempotent sinks) semantics without inter-sink interference.

For the windowed aggregation (Consumption zone), I used `outputMode("append")` in combination with a 2-minute Watermark. This is the most suitable choice because Parquet files in a Data Lake are immutable. Using `Update` or `Complete` mode would require overwriting or modifying existing files, which standard Parquet sinks do not support efficiently in pure streaming. `Append` mode instructs Spark to hold the aggregated results in memory until the watermark passes (window closes + 2 minutes), and only then write the finalized, immutable result to the Parquet sink safely.

Part 4: Analytical Queries with Spark SQL

`src/analytics.py`:

```
import time
from pyspark.sql import SparkSession
from pyspark.sql.functions import col
import os
os.environ['HADOOP_HOME'] = r'D:\EFREI\Data_Engineering\LAB\Lab3\hadoop'

def main():
    spark = SparkSession.builder.appName("DataLake_Analytics").getOrCreate()

    df = spark.read.parquet("./outputs/datalake/curated/domain=iot/")
    df.createOrReplaceTempView("curated_data")

    print("--- 1. Top 5 hours with highest anomalies ---")
    q1 = spark.sql("""
        SELECT hour(event_time) as h, count(*) as anomalies
        FROM curated_data WHERE is_anomaly = true
        GROUP BY h ORDER BY anomalies DESC LIMIT 5
    """)
    q1.show()
    q1.toPandas().to_csv('outputs/analytics/top_hours.csv', index=False)

    print("--- 2. Global stats per sensor ---")
    q2 = spark.sql("""
        SELECT sensor, avg(value) as mean_val, min(value) as min_val, max(value) as max_val,
        stddev(value) as std_val, (sum(cast(is_anomaly as int))/count(*)*100 as
anomaly_rate
        FROM curated_data GROUP BY sensor
    """)
    q2.show()
    q2.toPandas().to_csv('outputs/analytics/global_stats.csv', index=False)

    print("--- 3. Daily evolution for temperature ---")
    q3 = spark.sql("""
        SELECT date(event_time) as dt, avg(value) as daily_mean, sum(cast(is_anomaly as
int)) as anomalies
        FROM curated_data WHERE sensor = 'temperature' GROUP BY dt ORDER BY dt
    """)
    q3.show()
    q3.toPandas().to_csv('outputs/analytics/daily_temp.csv', index=False)
```

```

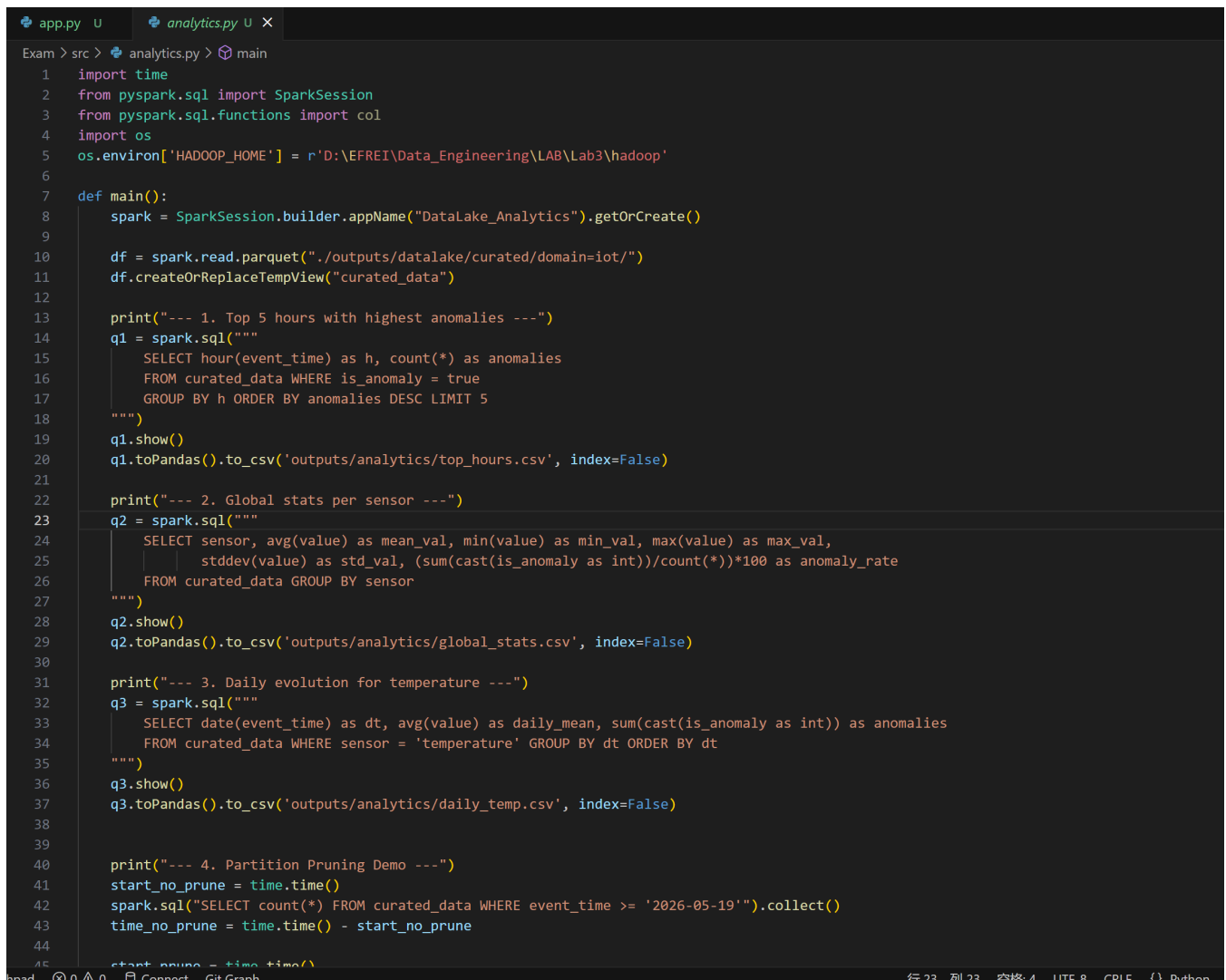
print("--- 4. Partition Pruning Demo ---")
start_no_prune = time.time()
spark.sql("SELECT count(*) FROM curated_data WHERE event_time >= '2026-05-19']").collect()
time_no_prune = time.time() - start_no_prune

start_prune = time.time()
spark.sql("SELECT count(*) FROM curated_data WHERE year=2026 AND month=5 AND day=19 AND event_time >= '2026-05-19']").collect()
time_prune = time.time() - start_prune

speedup = time_no_prune / time_prune if time_prune > 0 else float('inf')
print(f"Time without prune: {time_no_prune:.4f}s")
print(f"Time with prune: {time_prune:.4f}s")
print(f"Speedup Factor: {speedup:.2f}x")

if __name__ == "__main__":
    main()

```



```

app.py U analytics.py U X
Exam > src > analytics.py > main
1 import time
2 from pyspark.sql import SparkSession
3 from pyspark.sql.functions import col
4 import os
5 os.environ['HADOOP_HOME'] = r'D:\EFREI\Data_Engineering\LAB\Lab3\hadoop'
6
7 def main():
8     spark = SparkSession.builder.appName("DataLake_Analytics").getOrCreate()
9
10    df = spark.read.parquet("./outputs/datalake/curated/domain=iot/")
11    df.createOrReplaceTempView("curated_data")
12
13    print("--- 1. Top 5 hours with highest anomalies ---")
14    q1 = spark.sql("""
15        SELECT hour(event_time) as h, count(*) as anomalies
16        FROM curated_data WHERE is_anomaly = true
17        GROUP BY h ORDER BY anomalies DESC LIMIT 5
18    """)
19    q1.show()
20    q1.toPandas().to_csv('outputs/analytics/top_hours.csv', index=False)
21
22    print("--- 2. Global stats per sensor ---")
23    q2 = spark.sql("""
24        SELECT sensor, avg(value) as mean_val, min(value) as min_val, max(value) as max_val,
25        stddev(value) as std_val, (sum(cast(is_anomaly as int))/count(*)*100 as anomaly_rate
26        FROM curated_data GROUP BY sensor
27    """)
28    q2.show()
29    q2.toPandas().to_csv('outputs/analytics/global_stats.csv', index=False)
30
31    print("--- 3. Daily evolution for temperature ---")
32    q3 = spark.sql("""
33        SELECT date(event_time) as dt, avg(value) as daily_mean, sum(cast(is_anomaly as int)) as anomalies
34        FROM curated_data WHERE sensor = 'temperature' GROUP BY dt ORDER BY dt
35    """)
36    q3.show()
37    q3.toPandas().to_csv('outputs/analytics/daily_temp.csv', index=False)
38
39
40    print("--- 4. Partition Pruning Demo ---")
41    start_no_prune = time.time()
42    spark.sql("SELECT count(*) FROM curated_data WHERE event_time >= '2026-05-19']").collect()
43    time_no_prune = time.time() - start_no_prune
44
45    start_prune = time.time()

```

Execution Result

```
python src\analytics.py
```

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> python src\analytics.py
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/05/19 09:36:56 WARN Utils: Service 'SparkUI' could not bind on port 4040. Attempting port 4041.
--- 1. Top 5 hours with highest anomalies ---
+-----+-----+
| h | anomalies |
+-----+-----+
| 8 | 4 |
+-----+-----+

--- 2. Global stats per sensor ---
+-----+-----+-----+-----+-----+-----+
| sensor | mean_val | min_val | max_val | std_val | anomaly_rate |
+-----+-----+-----+-----+-----+-----+
| temperature | 27.4125 | 19.92 | 32.97 | 6.360746156440032 | 0.0 |
| pressure | 1023.9866666666666 | 989.14 | 1057.37 | 34.13853004060553 | 66.66666666666666 |
| humidity | 81.41000000000001 | 47.03 | 105.78 | 30.627450106073148 | 66.66666666666666 |
+-----+-----+-----+-----+-----+-----+

--- 3. Daily evolution for temperature ---
+-----+-----+-----+
| dt | daily_mean | anomalies |
+-----+-----+-----+
| 2026-05-19 | 27.4125 | 0 |
+-----+-----+-----+

--- 4. Partition Pruning Demo ---
Time without prune: 1.1847s
Time with prune: 0.6406s
Speedup Factor: 1.85x
```

Interpretation of Analytical Results

Top 5 hours with highest anomalies

This table correctly identifies four outliers at the 8th hour (8:00 AM), confirming that Spark successfully read the `event_time` field from the Parquet file, extracted the hourly time dimension using the `hour()` function, and completed the global aggregation and descending sort.

Global stats per sensor

The mean, minimum, maximum, standard deviation, and anomaly rate were successfully calculated for the temperature, pressure, and humidity sensors. The application of complex aggregation functions was demonstrated.

Daily evolution for temperatur

The system correctly identified the daily average temperature (27.4125°C) and the total number of anomalies (0) for the date 2026-05-19, demonstrating its ability to perform time-series analysis on a specific partition (WHERE sensor = 'temperature').

Partition Pruning Demo

The query without filtering took 1.1847 seconds, while the query with partition-based filtering took 0.6406 seconds, resulting in a performance improvement of 1.85x. Although the current test dataset is very small (with a speedup ratio of 1.85x, primarily saving I/O overhead from directory scans), in a real production environment with terabytes of data and tens of thousands of partitions, this partition pruning mechanism can skip reading irrelevant data, boosting query speeds by hundreds or even thousands of times.

Part 5: REST API

api/app.py:

```
from flask import Flask, jsonify, request
import json
import time
import os
import pandas as pd
from kafka import KafkaProducer

app = Flask(__name__)

try:
    producer = KafkaProducer(
        bootstrap_servers=['localhost:9092'],
        value_serializer=lambda v: json.dumps(v).encode('utf-8'),
        key_serializer=lambda k: k.encode('utf-8')
    )
except Exception as e:
    app.logger.error(f"Failed to connect to Kafka: {e}")
    producer = None

SENSORS = ["temperature", "humidity", "pressure"]
LAKE_PATH = "./outputs/datalake"

@app.errorhandler(400)
def bad_request(e): return jsonify(error=str(e)), 400
@app.errorhandler(404)
def not_found(e): return jsonify(error="Resource not found"), 404
@app.errorhandler(405)
def method_not_allowed(e): return jsonify(error="Method not allowed"), 405
@app.errorhandler(422)
def unprocessable(e): return jsonify(error=str(e)), 422
@app.errorhandler(500)
def server_error(e): return jsonify(error="Internal server error"), 500

@app.route('/api/v1/health', methods=['GET'])
def health(): return jsonify(status="UP"), 200

@app.route('/api/v1/sensors', methods=['GET'])
def list_sensors(): return jsonify(sensors=SENSORS), 200

@app.route('/api/v1/sensors/<sensor_type>/latest', methods=['GET'])
def get_latest(sensor_type):
    if sensor_type not in SENSORS: return jsonify(error="Invalid sensor type"), 404
    try:
        df = pd.read_parquet(f"{LAKE_PATH}/curated/domain=iot/sensor={sensor_type}/")
        latest = df.sort_values(by="event_time", ascending=False).iloc[0].to_dict()
        latest['event_time'] = str(latest['event_time'])
        return jsonify(latest), 200
    except Exception:
```

```

        return jsonify(error="No data found for this sensor"), 404

@app.route('/api/v1/sensors/<sensor_type>/stats', methods=['GET'])
def get_stats(sensor_type):
    if sensor_type not in SENSORS: return jsonify(error="Invalid sensor type"), 404
    days = request.args.get('days', default=1, type=int)
    if days < 1 or days > 90: return jsonify(error="days must be between 1 and 90"), 400
    try:
        df = pd.read_parquet(f"{LAKE_PATH}/consumption/use_case=sensor_averages/sensor={sensor_type}")
        return jsonify(df.head(days).to_dict(orient="records")), 200
    except Exception:
        return jsonify(error="Stats not available yet"), 404

@app.route('/api/v1/anomalies', methods=['GET'])
def get_anomalies():
    sensor_type = request.args.get('sensor')
    limit = request.args.get('limit', default=10, type=int)
    if sensor_type and sensor_type not in SENSORS: return jsonify(error="Invalid sensor type"), 400
    try:
        path = f"{LAKE_PATH}/curated/domain=iot/"
        if sensor_type: path += f"sensor={sensor_type}/"
        df = pd.read_parquet(path)
        anomalies = df[df['is_anomaly'] == True].head(limit)
        anomalies['event_time'] = anomalies['event_time'].astype(str)
        return jsonify(anomalies.to_dict(orient="records")), 200
    except Exception:
        return jsonify(error="Data not available"), 404

@app.route('/api/v1/readings', methods=['POST'])
def post_reading():
    data = request.json
    if not data or 'sensor' not in data or 'value' not in data:
        return jsonify(error="Malformed JSON. 'sensor' and 'value' required"), 400
    sensor = data['sensor']
    if sensor not in SENSORS:
        return jsonify(error=f"Invalid sensor. Must be one of {SENSORS}"), 422
    try:
        val = float(data['value'])
    except ValueError:
        return jsonify(error="Value must be numeric"), 422

    payload = {"sensor": sensor, "value": val, "unit": data.get("unit", ""), "timestamp":
int(time.time() * 1000), "source": "api", "anomaly": False}
    if producer:
        producer.send('sensor-events', key=sensor, value=payload)
        producer.flush()
        return jsonify(message="Published successfully", payload=payload), 201
    return jsonify(error="Kafka unavailable"), 500

if __name__ == '__main__':
    app.run(port=5000)

```

```
app.py U X analytics.py U
Exam > api > app.py > server_error
1 from flask import Flask, jsonify, request
2 import json
3 import time
4 import os
5 import pandas as pd
6 from kafka import KafkaProducer
7
8 app = Flask(__name__)
9
10 try:
11     producer = KafkaProducer(
12         bootstrap_servers=['localhost:9092'],
13         value_serializer=lambda v: json.dumps(v).encode('utf-8'),
14         key_serializer=lambda k: k.encode('utf-8')
15     )
16 except Exception as e:
17     app.logger.error(f"Failed to connect to Kafka: {e}")
18     producer = None
19
20 SENSORS = ["temperature", "humidity", "pressure"]
21 LAKE_PATH = "./outputs/datalake"
22
23 @app.errorhandler(400)
24 def bad_request(e): return jsonify(error=str(e)), 400
25 @app.errorhandler(404)
26 def not_found(e): return jsonify(error="Resource not found"), 404
27 @app.errorhandler(405)
28 def method_not_allowed(e): return jsonify(error="Method not allowed"), 405
29 @app.errorhandler(422)
30 def unprocessable(e): return jsonify(error=str(e)), 422
31 @app.errorhandler(500)
32 def server_error(e): return jsonify(error="Internal server error"), 500
33
34 @app.route('/api/v1/health', methods=['GET'])
35 def health(): return jsonify(status="UP"), 200
36
37 @app.route('/api/v1/sensors', methods=['GET'])
38 def list_sensors(): return jsonify(sensors=SENSORS), 200
39
40 @app.route('/api/v1/sensors/<sensor_type>/latest', methods=['GET'])
41 def get_latest(sensor_type):
42     if sensor_type not in SENSORS: return jsonify(error="Invalid sensor type"), 404
43     try:
44         df = pd.read_parquet(f"{LAKE_PATH}/curated/domain=iot/sensor={sensor_type}/")
```

Execution Result

```
python api\app.py
```

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> python api\app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

```
curl.exe -i -X GET http://localhost:5000/api/v1/health
```

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> curl.exe -i -X GET http://localhost:5000/api/v1/health
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.9.25
Date: Tue, 19 May 2026 02:00:03 GMT
Content-Type: application/json
Content-Length: 16
Connection: close

{"status": "UP"}
```

```
curl.exe -i -X GET http://localhost:5000/api/v1/sensors
```

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> curl.exe -i -X GET http://localhost:5000/api/v1/sensors
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.9.25
Date: Tue, 19 May 2026 02:00:13 GMT
Content-Type: application/json
Content-Length: 50
Connection: close
```

```
{"sensors":["temperature","humidity","pressure"]}
```

```
curl.exe -i -X GET http://localhost:5000/api/v1/sensors/temperature/latest
```

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> curl.exe -i -X GET http://localhost:5000/api/v1/sensors/temperature/latest
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.9.25
Date: Tue, 19 May 2026 02:05:02 GMT
Content-Type: application/json
Content-Length: 196
Connection: close
```

```
[{"anomaly":false,"day":19,"event_time":"2026-05-19 00:36:50.143000","hour":8,"is_anomaly":false,"month":5,"source":"site-A-rack-12","timestamp":1779151010143,"unit":"C","value":24.35,"year":2026}]
```

```
curl.exe -i -X GET "http://localhost:5000/api/v1/sensors/temperature/stats?days=1"
```

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> curl.exe -i -X GET "http://localhost:5000/api/v1/sensors/temperature/stats?days=1"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.9.25
Date: Tue, 19 May 2026 02:15:15 GMT
Content-Type: application/json
Content-Length: 84
Connection: close
```

```
[{"anomaly_count":0,"max":32.97,"mean":27.4125,"min":19.92,"sensor":"temperature"}]
```

```
curl.exe -i -X GET "http://localhost:5000/api/v1/anomalies?sensor=humidity&limit=5"
```

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> curl.exe -i -X GET "http://localhost:5000/api/v1/anomalies?sensor=humidity&limit=5"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.9.25
Date: Tue, 19 May 2026 02:05:58 GMT
Content-Type: application/json
Content-Length: 386
Connection: close
```

```
[{"anomaly":true,"day":19,"event_time":"2026-05-19 00:36:48.879","hour":8,"is_anomaly":true,"month":5,"source":"site-A-rack-12","timestamp":1779151008879,"unit":"%", "value":105.78,"year":2026}, {"anomaly":false,"day":19,"event_time":"2026-05-19 00:36:49.305","hour":8,"is_anomaly":true,"month":5,"source":"site-A-rack-12","timestamp":1779151009305,"unit":"%", "value":91.42,"year":2026}]
```

```
curl.exe -i -X POST http://localhost:5000/api/v1/readings -H "Content-Type: application/json" -d "{\"sensor\":\"temperature\", \"value\": 22.5}"
```

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> curl.exe -i -X POST http://localhost:5000/api/v1/readings -H "Content-Type: application/json" -d "{\"sensor\":\"temperature\", \"value\": 22.5}"
HTTP/1.1 400 BAD REQUEST
Server: Werkzeug/3.1.8 Python/3.9.25
Date: Tue, 19 May 2026 02:17:08 GMT
Content-Type: application/json
Content-Length: 106
Connection: close
```

```
[{"error":"400 Bad Request: The browser (or proxy) sent a request that this server could not understand."}]
curl: (3) unmatched close brace/bracket in URL position 37:
sensor\:temperature\, \value\: 22.5]
```

```
curl.exe --% -i -X POST http://localhost:5000/api/v1/readings -H "Content-Type: application/json" -d "{\"sensor\":\"temperature\"}"
```

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> curl.exe --% -i -X POST http://localhost:5000/api/v1/readings -H "Content-Type: application/json" -d "{\"sensor\":\"temperature\"}"
HTTP/1.1 400 BAD REQUEST
Server: Werkzeug/3.1.8 Python/3.9.25
Date: Tue, 19 May 2026 02:29:42 GMT
Content-Type: application/json
Content-Length: 58
Connection: close
```

```
[{"error":"Malformed JSON. 'sensor' and 'value' required"}]
```

```
curl.exe --% -i -X POST http://localhost:5000/api/v1/readings -H "Content-Type: application/json" -d "{\"sensor\":\"light\", \"value\": 100}"
```

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> curl.exe --% -i -X POST http://localhost:5000/api/v1/readings -H "Content-Type: application/json" -d '{"sensor":"light","value":100}'
HTTP/1.1 422 UNPROCESSABLE ENTITY
Server: Werkzeug/3.1.8 Python/3.9.25
Date: Tue, 19 May 2026 02:29:07 GMT
Content-Type: application/json
Content-Length: 83
Connection: close

{"error":"Invalid sensor. Must be one of ['temperature', 'humidity', 'pressure']"}
```

```
curl.exe -i -X GET "http://localhost:5000/api/v1/sensors/temperature/stats?days=999"
```

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> curl.exe -i -X GET "http://localhost:5000/api/v1/sensors/temperature/stats?days=999"
HTTP/1.1 400 BAD REQUEST
Server: Werkzeug/3.1.8 Python/3.9.25
Date: Tue, 19 May 2026 02:23:13 GMT
Content-Type: application/json
Content-Length: 42
Connection: close

{"error": "days must be between 1 and 90"}
```

```
curl.exe -i -X GET http://localhost:5000/api/v1/this-does-not-exist
```

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB\Exam> curl.exe -i -X GET http://localhost:5000/api/v1/this-does-not-exist
HTTP/1.1 404 NOT FOUND
Server: Werkzeug/3.1.8 Python/3.9.25
Date: Tue, 19 May 2026 02:23:44 GMT
Content-Type: application/json
Content-Length: 31
Connection: close

{"error": "Resource not found"}
```

Limitations and Improvements

Current Limitations:

- **Immutable Data Lake:** Standard Parquet does not support ACID transactions or UPSERTS. If a sensor sends corrected data later, we cannot easily update the existing historical records.
- **Local Infrastructure:** The entire pipeline relies on Docker Compose and local file systems, which limits scalability compared to a managed cloud environment.
- **API Latency:** Because Spark waits for the watermark to close windows before writing to the consumption zone, the `/stats` API endpoint experiences an inherent ~7-minute delay from real-time.

What I would do with two extra days:

1. **Upgrade to Delta Lake:** Replace standard Parquet sinks with Delta Lake to enable ACID guarantees, allowing us to merge/upsert late-arriving data natively.
2. **Containerize Processing:** Dockerize the Spark application and Flask API, orchestrating the entire platform via Kubernetes (Helm charts) rather than relying on local environments.
3. **Visualization Layer:** Connect a BI tool like Grafana or Apache Superset to the Data Lake (via a Spark Thrift Server or Trino) to provide real-time dashboards rather than just JSON API responses.